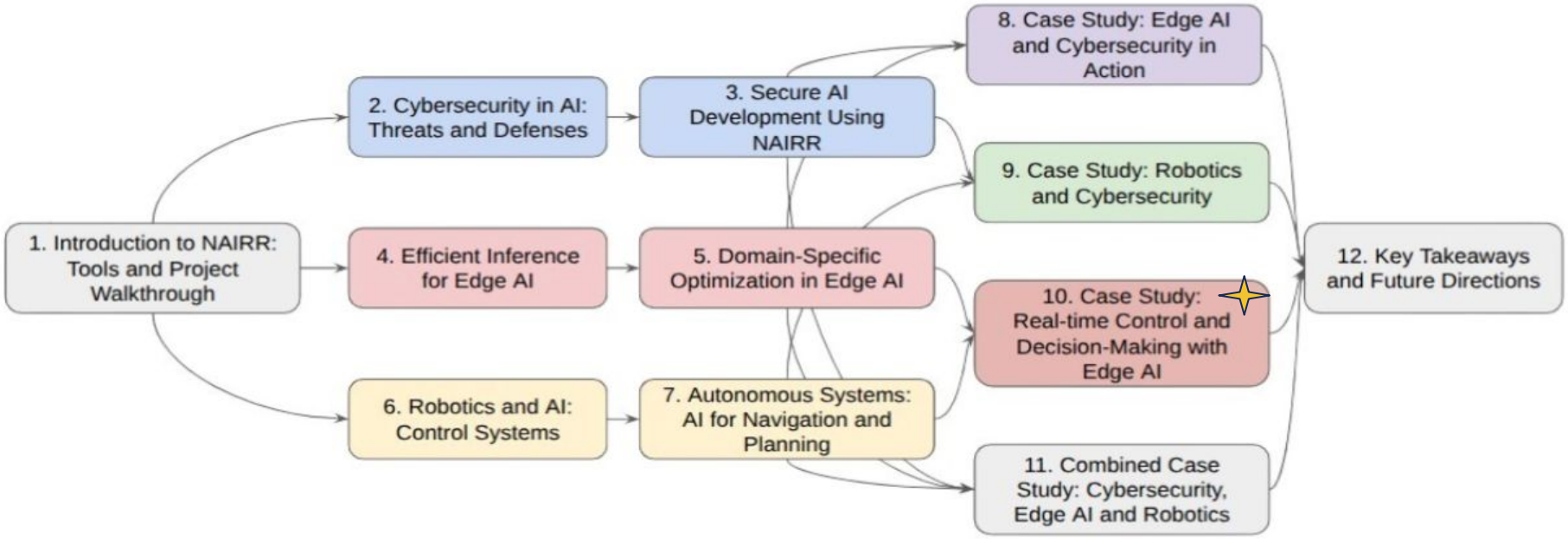
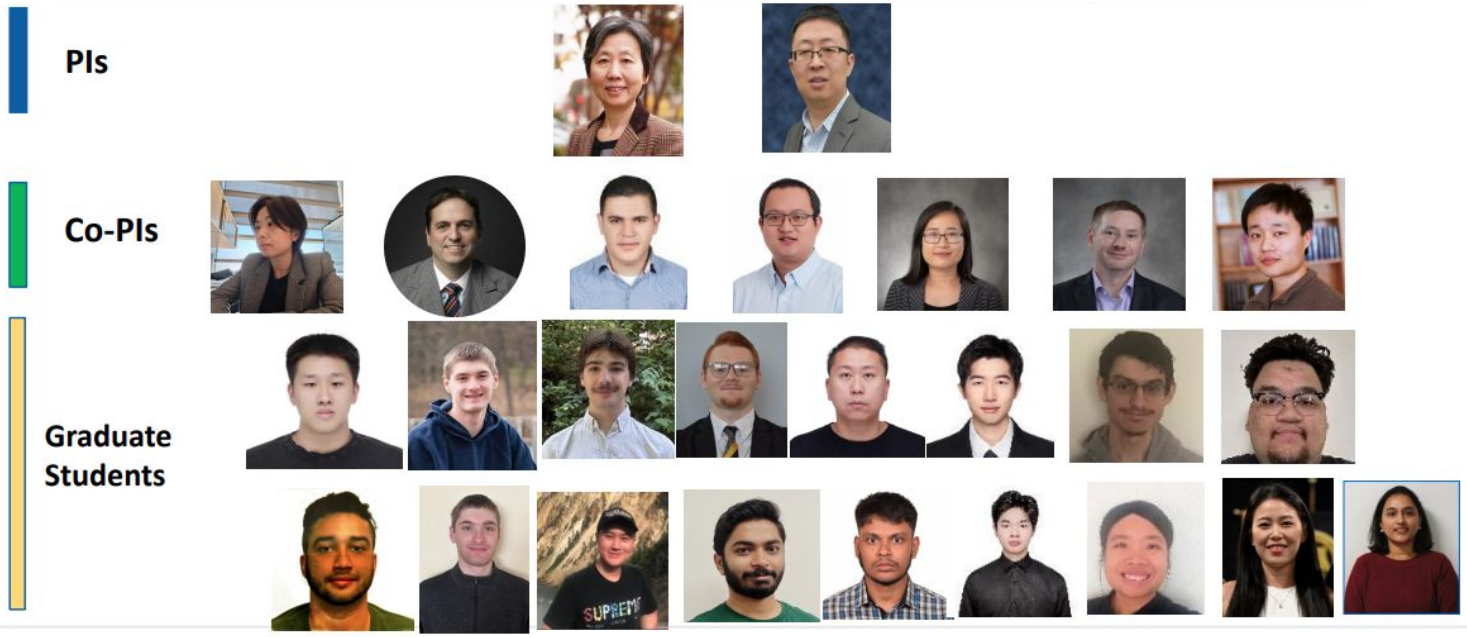
MODULE 5.3 • NAIRR WORKSHOP

Case Study: Real-time Control & Decision-Making with Edge AI

Core Topics Covered

- Edge-AI Agents
- Edge-Decision Making
- Imitation Learning
- Intelligence Offloading


1. Introduction to NAIRR: Tools and Project Walkthrough
2. Cybersecurity in AI: Threats and Defenses
3. Secure AI Development Using NAIRR
4. Efficient Inference for Edge AI
5. Domain-Specific Optimization in Edge AI
6. Robotics and AI: Control Systems
7. Autonomous Systems: AI for Navigation and Planning
8. Case Study: Edge AI and Cybersecurity in Action
9. Case Study: Robotics and Cybersecurity
10. Case Study: Real-time Control and Decision-Making with Edge AI
11. Combined Case Study: Cybersecurity, Edge AI and Robotics
12. Key Takeaways and Future Directions


PIs
Co-PIs
Graduate Students

Meet the Team

Dr. Mohammad Wardat

Assistant Professor
Oakland University

- Research focuses on software engineering, artificial intelligence
- Focuses both on academic research and practical implementation

Muhammad Anas Raza

PhD Student
Oakland University

- Research focuses on software reliability

Niful Islam

PhD Student
Oakland University

- Research focuses on Large Language Models and AI Agents

MODULE 5.3 • NAIRR WORKSHOP

Case Study: Real-time Control & Decision-Making with Edge AI

Core Topics Covered

- Edge-AI Agents
- Edge-Decision Making
- Imitation Learning
- Intelligence Offloading

Pre-workshop Survey

Scan to participate

Agenda

01

Why need Edge AI

We will discuss the motivations behind using edge AI, and the paradigm shift towards edge AI

02

Foundations of Edge AI

Core principles for building responsive, efficient, and intelligent edge AI systems

03

Case studies on Edge AI

Four case studies with hands-on experiments on edge chatbot, Real-Time Control, Edge Robotics, and Decision-Making

Motivation: Systems Under Constraints

Hardware latency in robotics can destabilize control loops.

Real-world sensor noise creates gaps between simulation and reality.

GenAI traffic bursts often lead to severe GPU congestion.

We need adaptive hybrid systems to handle these variables.

Why This Actually Matters

The 250ms Blink

Human reaction time is about 250 milliseconds. A round trip to a distant cloud server can eat up half of that before a robot even starts responding.

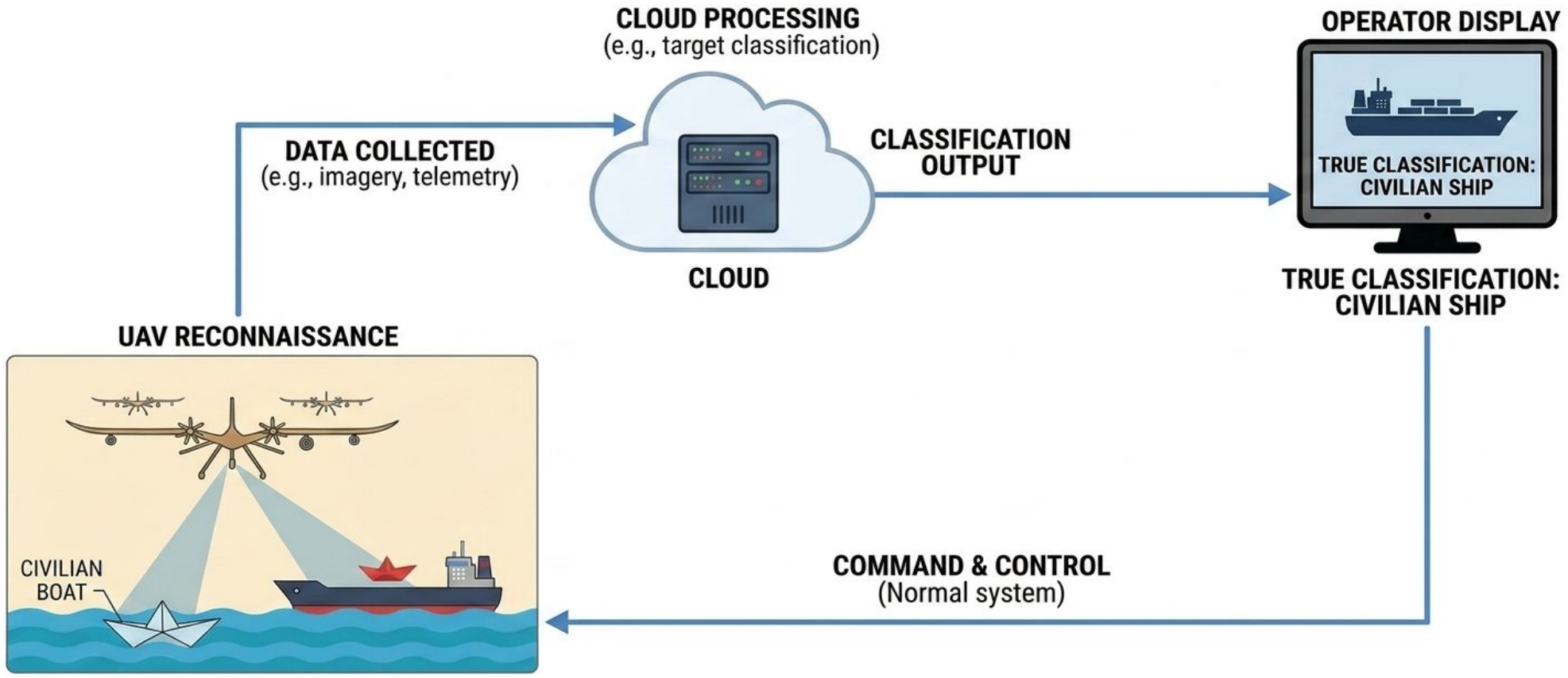
Apollo in Your Pocket

The Apollo 11 guidance computer ran on about 4KB of RAM. Devices running today's edge AI models are millions of times more powerful, and small enough to fit in a pocket.

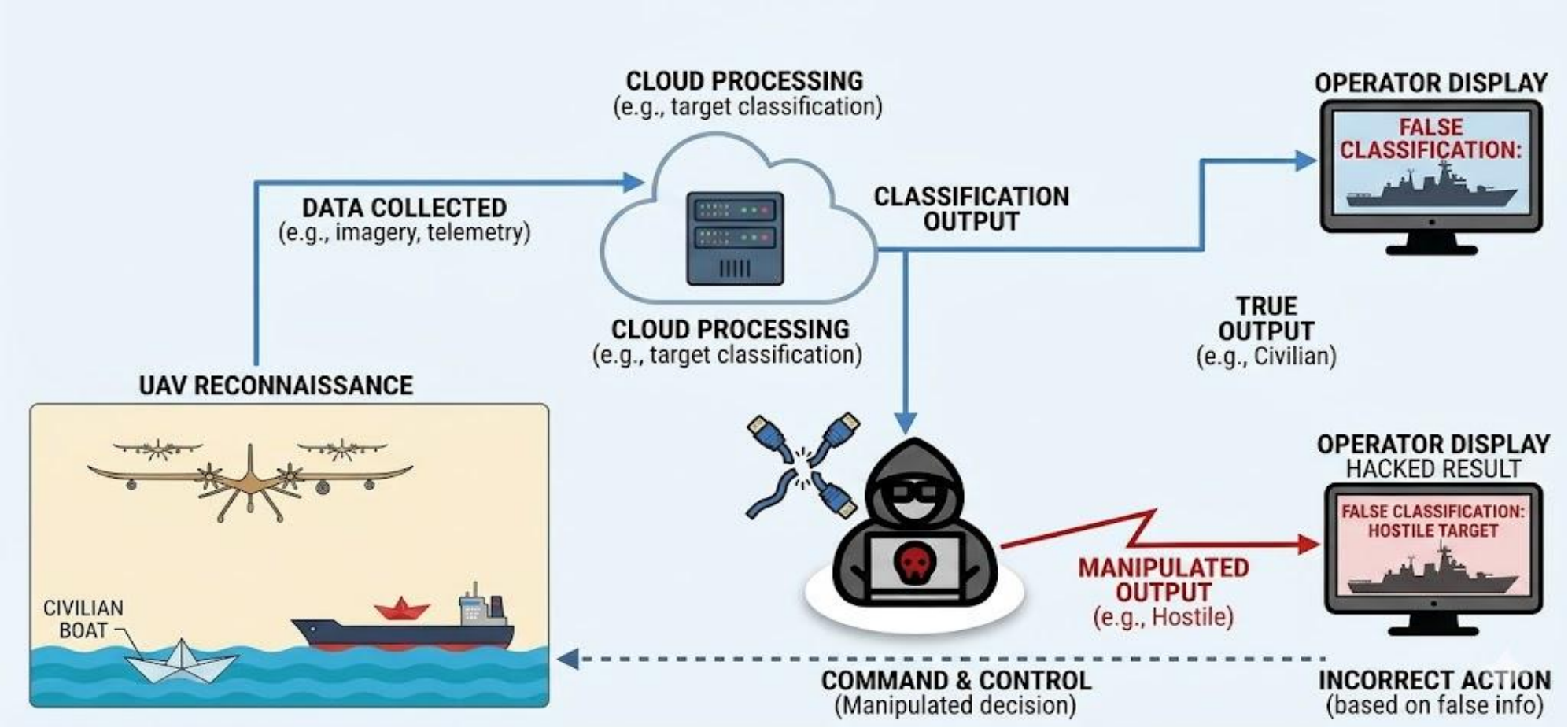
$150 Billion Question

The U.S. Department of Energy estimates outages and instability cost the American economy about $150 billion every year, exactly the kind of failure real-time edge systems are built to prevent.

Example: Cloud Data Processing



Example: Manipulation of Decisions in Cloud Processing



The Paradigm Shift to Edge Computing

Cloud Latency

Cloud computing introduces unavoidable network latency and bandwidth bottlenecks.

Edge Processing

Pushes processing directly to the data source such as sensors, robots, and local gateways.

Sub-Millisecond Reaction

Crucial for mission-critical applications requiring immediate, real-time response times.

Localized Autonomy

Reduces reliance on continuous network connectivity, enabling independent local operations.

Defining Real-Time Control

Dynamic Monitoring

Systems that continuously monitor and adjust dynamic environments (e.g., autonomous driving).

The Control Loop

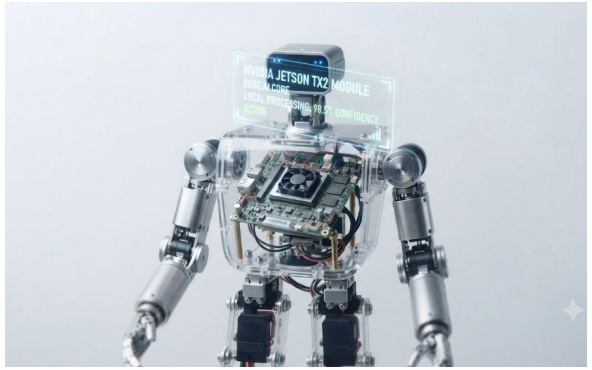
Relies on a strict, continuous "Sense → Compute → Act" loop for instant adjustments.

Latency Danger

Delays in this loop can cause severe system instability, safety hazards, or hardware failure.

AI Augmentation

Traditional physics-based controllers (like PID) are now being augmented by AI for predictive control.



Edge Control in Action

Onboard AI systems performing real-time inference & adjustments.

The Environmental Constraints

Compute Limits

Edge devices (e.g., Raspberry Pi, onboard compute) have strict memory and thermal caps.

Sensor Reality

Real-world sensors (LiDAR, telemetry) produce highly noisy, high-variance data compared to simulations.

Network Jitter

Fluctuating network speeds make cloud-reliant control highly unpredictable.

The Solution

AI models must be aggressively optimized (via feature abstraction or quantization) to survive on the edge.

Intelligent Decision-Making & Offloading

Edge Compute Limits

Not every complex task can or should be processed locally due to limited resource caps.

Dynamic Routing

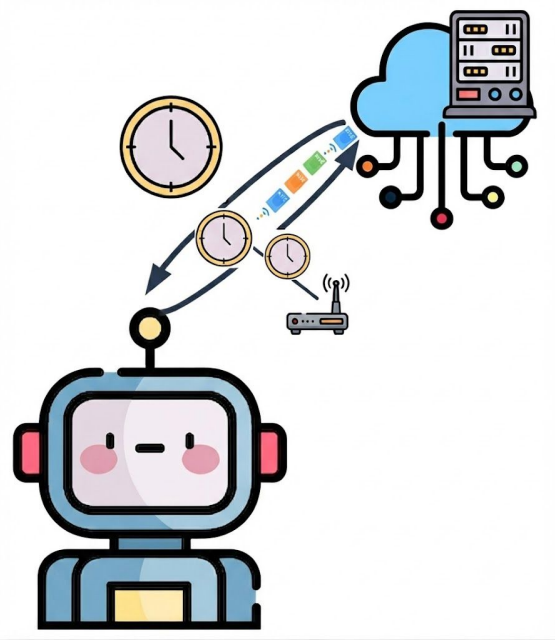
Systems must continuously decide whether to compute locally or offload tasks to the cloud.

Real-Time Awareness

Decisions require instant monitoring of local GPU loads, queue sizes, and network status.

SLA vs. Cost Balance

The ultimate goal is balancing strict service level agreements (SLAs) against egress costs.



Transitioning to the Case Studies

How do we solve these foundational challenges in practice?

Case 1: Edge Chatbot

On-Device Interaction

Uses compact Large Language Model for sensitive question answering.

Case 2: Edge Robotics

Sensor Compression

Compressing raw sensor perception into lightweight, real-time motor commands.

Case 3: Real-Time Control

Energy Prediction

Appliances Energy Prediction: Edge AI & Time-Series Forecasting.

Case 4: Decision-Making

Intelligent Offloading

Using neural networks to dynamically route GenAI tasks across a cluster.

CASE 01

Using compact Large Language Model for Sensitive Question Answering

Why Keep It Small and Local

Security

$9.77 Million Mistake

The average healthcare data breach cost $9.77 million in 2024, the highest of any industry for the 14th year running (IBM/Ponemon).

Memory

100x Smaller, Still Useful

Frontier chatbots run into the hundreds of billions of parameters. Gemma-2-2B, used here, is roughly 100x smaller, and once quantized, fits comfortably on a laptop.

Lock

Nothing Leaves the Room

Because the model runs entirely on-device, sensitive medical records never touch the cloud: no API call, no third-party server, no exposure.

LLM

01. Large Scale Training

LLMs are neural networks trained on vast amount of texts

02. Next Word Prediction

LLMs take texts as input and generates the next word as output

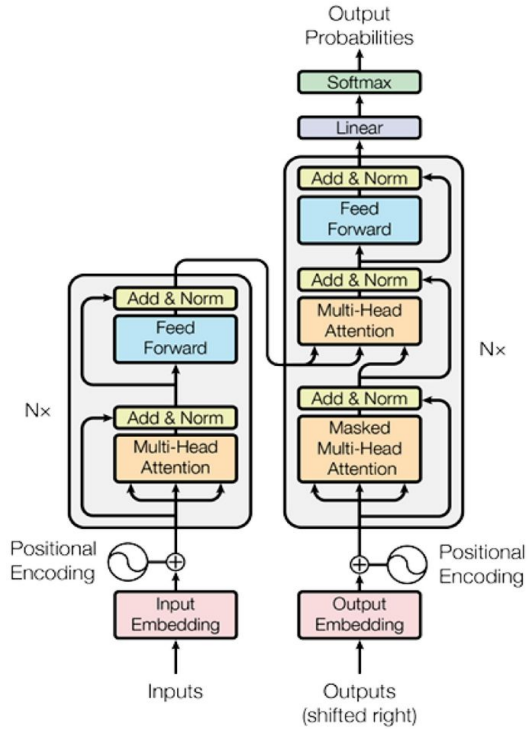
03. Billions of Parameters

Large LLMs have billions of parameters

04. Billions of Parameters

LLMs answer questions, summarize text, and write code

ARCHITECTURE DIAGRAM



Visual representation of the transformer architecture.

LLM Used for the Study

01. Gemma-2-2B

Google's open-source language model with 2 billion parameters

02. Instruction-Tuned

Fine-tuned specifically to follow instructions and hold conversations

03. Quantization

The model's weights are compressed to 4-bit precision

Encoder Used for the Study

01. MiniLM
A lightweight encoder built for speed and efficiency

02. Output Dimensionality
Compresses input text into compact, dense vector embeddings of size 384

03. Layers
Uses just 6 transformer layers, making it fast and resource-friendly

03. Parameters
Totals 22.7M parameters, comparatively lower than other models

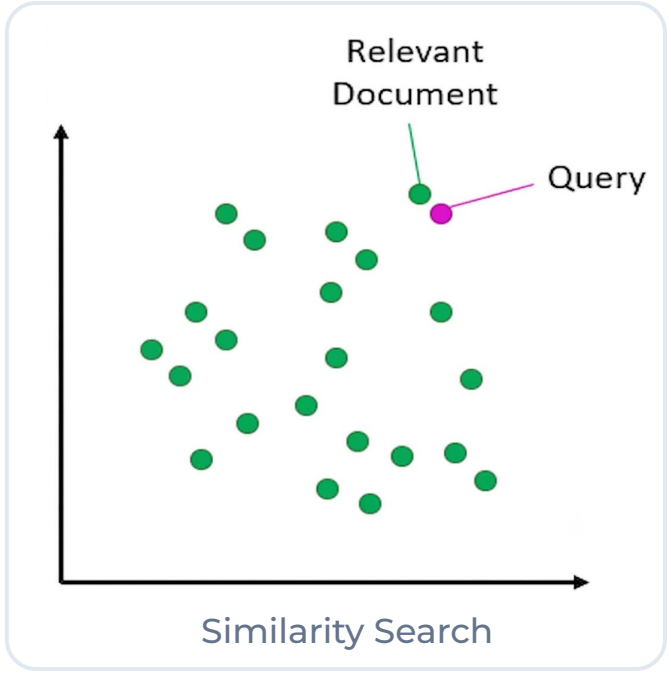
Similarity Search

01. Vector Comparison
Finds items whose embeddings are closest in vector space

02. Nearest Neighbors
Retrieves the most relevant matches by ranking items based on how close their vectors are to a query vector

03. FAISS
A library by Meta AI for efficient similarity search over dense vector embeddings

03. Indexing
Uses indexing techniques to quickly find the closest vectors without exhaustive comparison



Similarity Search

LLM Agents

01. Autonomous Systems

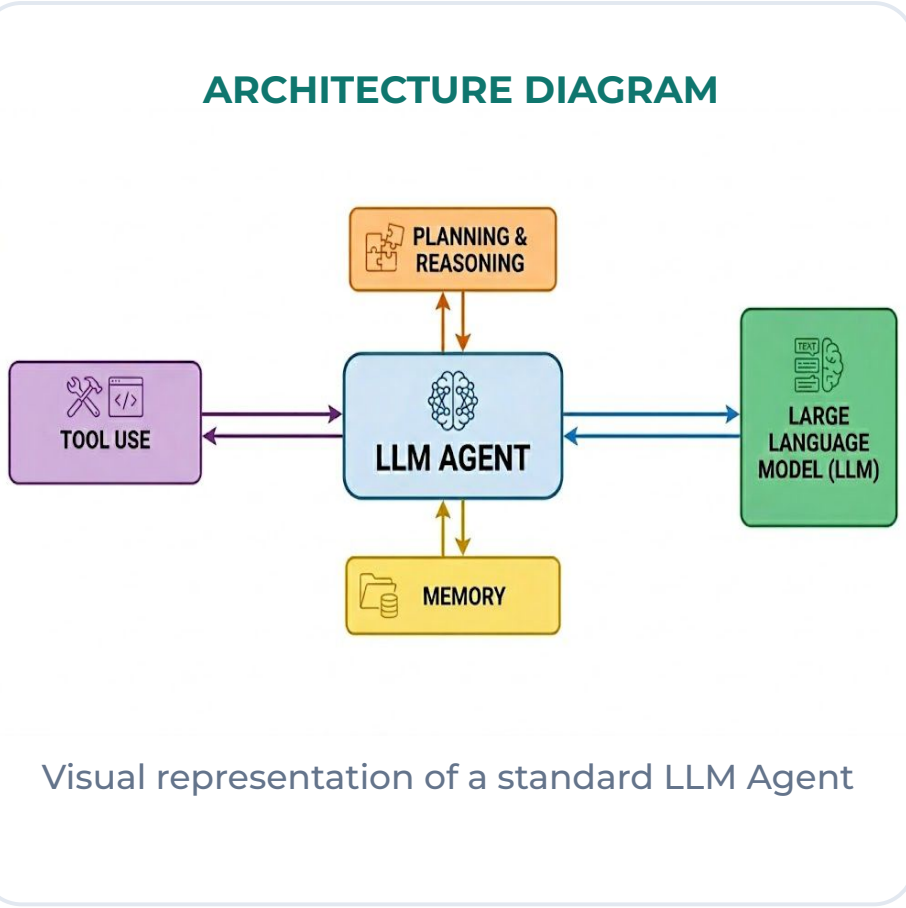
LLM agents are independent systems capable of taking actions

02. Four Core Components

It consists of four components: Agent code (LLM), memory, tool and planning

03. Broad Applications

Extensively used for multiple tasks like question answering, bug fixing, and labeling



Visual representation of a standard LLM Agent

RAG Agent

01. Document Retrieval

RAG combines a document retriever with an LLM to generate answers grounded in specific external data

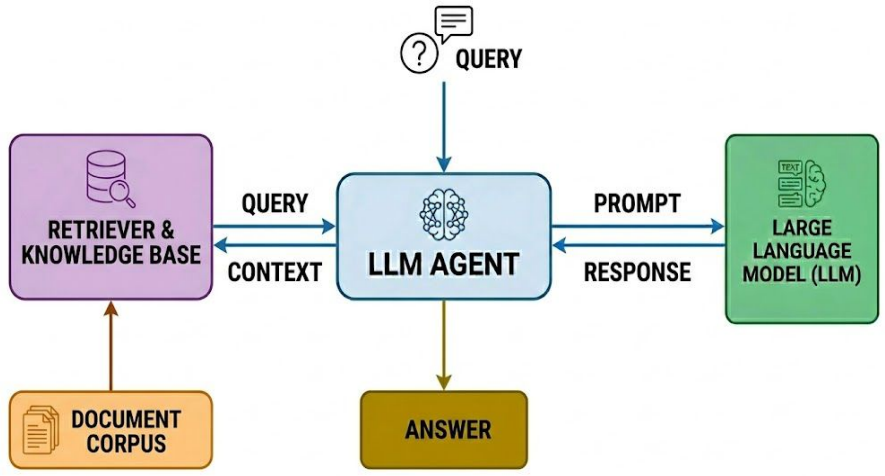
02. External Knowledge

It provides access to information outside the LLM's training set

03. Private Data Support

Can be used for secure, local processing of sensitive clinical records

ARCHITECTURE DIAGRAM



Visual representation of the RAG architecture.

Notebook: 01 - RAG System For Medical QA

- Uploads user’s medical documents in the system
- It reads the documents, splits into chunks and converts to vectors
- When the user asks questions, it retrieves similar segments and answers from it

Take It Further: Try This Next

Ollama + llama.cpp

Skip building your own inference stack. “ollama run gemma3” serves a quantized model locally in one command, the same GGUF format this case study's Gemma-2-2B could ship in.

Gemma 3 & Phi-4

Google's Gemma 3 and Microsoft's Phi-4 pick up where Gemma-2-2B leaves off: still small enough for a laptop, but closer to frontier quality on reasoning and instruction-following.

LlamaIndex / Haystack

Purpose-built RAG frameworks that handle chunking, embeddings, and citation tracking, so the next prototype takes hours, not weeks.

CASE 02

End-to-End Imitation Learning on Edge Devices

How Robots Learn Without a Rulebook

Toddler Logic

A toddler learns not to walk into a wall by watching, not by being handed an if-then rulebook. Imitation learning teaches robots the same way.

1 Million and Counting

Amazon passed one million deployed warehouse robots in 2025, a fleet including the Proteus robots featured in this case study, now nearly matching its 1.2 million-person warehouse workforce.

No Two Days Alike

The same aisle looks different by the hour as people and inventory move through it, exactly why fixed rules break and learned policies don't.

Notebook 02: - Objective: Behavioral Cloning

Goal

Train a robot to mimic an 'expert' agent's navigation tasks.

Strategy

Map raw Perception (LiDAR) directly to Action (Motor Control).

Advantage

Replaces brittle manual if-then rules with a fluid neural policy.

Notebook 02: - Background

01. Neural Network

Layers of connected neurons: input layer, hidden layers, and output layer.

02. Learns via Weights

Weights adjust during training to systematically reduce prediction errors over time.

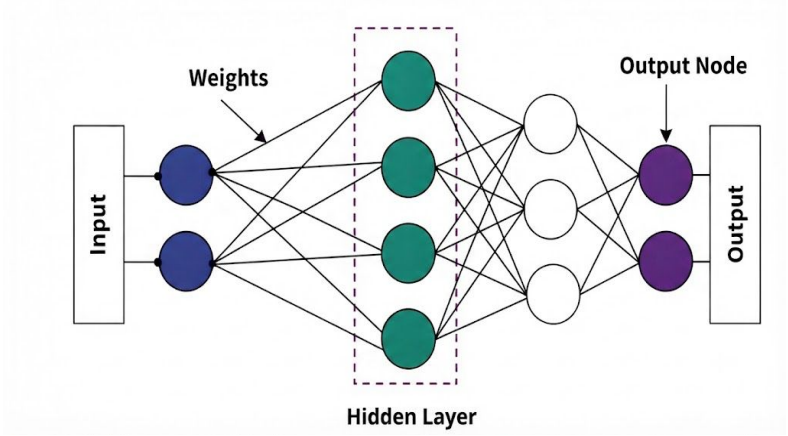
03. Activation Function

Adds non-linearity to outputs, enabling the network to learn highly complex patterns.

04. Multi-Layer Perceptron (MLP)

A structured stack of neural network layers that process data step-by-step.

ARCHITECTURE DIAGRAM



Visual representation of signal flow from the Input Layer, through Hidden Layers, to the final Output Node.

Notebook 02: - Background

Brittle Rules

Traditional "if-then" navigation rules often fail in complex, dynamic environments that the original programmer did not explicitly define.

Perception-Action Gap

Translating high-dimensional sensor data (like LiDAR) into precise motor commands manually is computationally expensive and error-prone.

Dynamic Obstacles

Static algorithms struggle to maintain fluid trajectories when faced with unpredictable spatial layouts and moving objects.

Notebook 02: - Sensor Abstraction (LiDAR)

Raw LiDAR

Provides 360+ points of high-dimensional, noisy depth data that is challenging to process directly on edge devices.

Binning Fix

Technical Fix: 'Binning' points into three structured spatial cones: Front, Left, and Right.

Edge Benefit

Minimizes raw sensor noise and significantly reduces computational overhead for efficient edge execution.

Notebook: 02 - Robust Data Pipeline

LiDAR Normalization

Normalized LiDAR distances (clipped at 5m) to prevent gradient explosion.

Data Splitting

Split by episodes (files) rather than randomized steps.

Why It Matters

Prevents the model from memorizing specific room layouts.

Notebook 02: - Baseline MLP Policy

Architecture

Lightweight 3-layer MLP optimized for low-latency embedded execution.

Robustness

Integrated Dropout (0.2) to mitigate stochastic sensor noise and failures.

Objective

Maps 3D inputs to 2D motor commands via Mean Squared Error (MSE) loss.

Notebook 02: - Residual Learning Policy

Architecture

Deep feature extractor utilizing **Skip Connections** to prevent signal degradation.

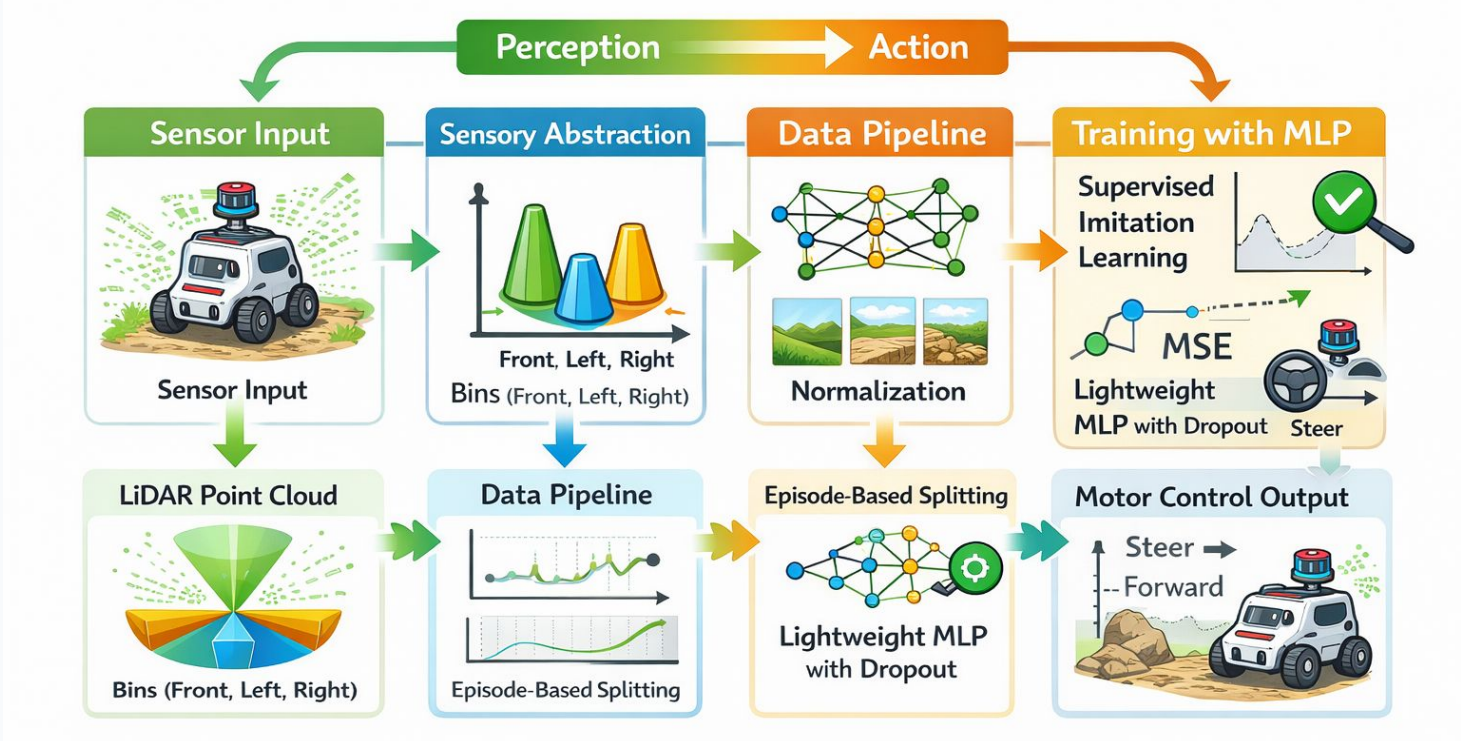
Stability

Employs **Batch Normalization** to handle varied sensor dynamic ranges.

Performance

Projects input to a **64-feature manifold** for complex obstacle negotiation.

Schematic Diagram



Case Study 02: Schematic Diagram

Notebook 02: - Validation Outcomes

RMSE Analysis

Evaluated overall system performance using quantitative RMSE analysis of predicted actions.

Behavioral Visuals

Verified obstacles result in correct, safe maneuvers through active visual testing.

Driving Consistency

Confirmed robust driving consistency across both smooth floors and rough surfaces.

Notebook 02: Industrial Use case

Use Case

Autonomous navigation for "Proteus" warehouse robots.

The Problem

Fixed-path robots get stuck in dynamic, human-filled spaces.

Solution

Imitation learning mimics human navigation and safety logic.

Result

Robots safely navigate crowded floors without manual resets.

Take It Further: Try This Next

LeRobot (Hugging Face)

An open-source library that packages data collection, ACT and Diffusion Policy training, and real-hardware deployment, taking a team from teleoperation to a trained policy in days.

SmolVLA & Pi0

Vision-Language-Action models under 500M parameters now generalize across tasks and objects the way this case study's MLP was trained to handle just one.

SO-100 / SO-101 Arms

Community-built robot arms costing a few hundred dollars have turned imitation learning from a research-lab exercise into a weekend project.

CASE 03

Real-Time Control and Decision-making with Edge

Notebook 03: - Objective: Edge-Style Forecasting

Razor-Thin Tolerance

The U.S. power grid must hold within roughly 0.05 Hz of 60 Hz. Drift too far for too long, and protective systems start disconnecting equipment automatically.

$150 Billion a Year

The Department of Energy estimates outages and grid instability cost the American economy about $150 billion annually, leaving little room for a slow reaction.

Minutes, Not Milliseconds

Unlike a robot's control loop, grid decisions often come with a few minutes of runway, exactly the gap this case study's forecasting models are built to fill.

Notebook 03: - Objective: Edge-Style Forecasting

Goal: Build a time-series forecasting pipeline using Ridge Regression, 1D-CNN, and GRU, plus an edge decision agent.

Application: Predict household appliance energy consumption and simulate real-time load shedding via an Edge Controller.

Constraint: Manage dynamic time-series patterns using windowed preprocessing and ensure reliable edge predictions.

Notebook 03: - Background

Grid Instability

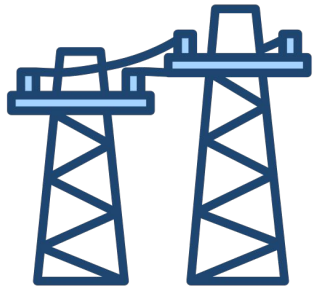
Residential peak loads are highly volatile and non-linear, threatening local grid stability.

The Latency Gap

Traditional cloud processing is too slow for the sub-second decisions required for real-time load shedding.

Edge Intelligence

An autonomous Edge Controller processes windowed data locally to simulate immediate, reliable load-shedding actions.



Notebook 03: - Background

Penalty

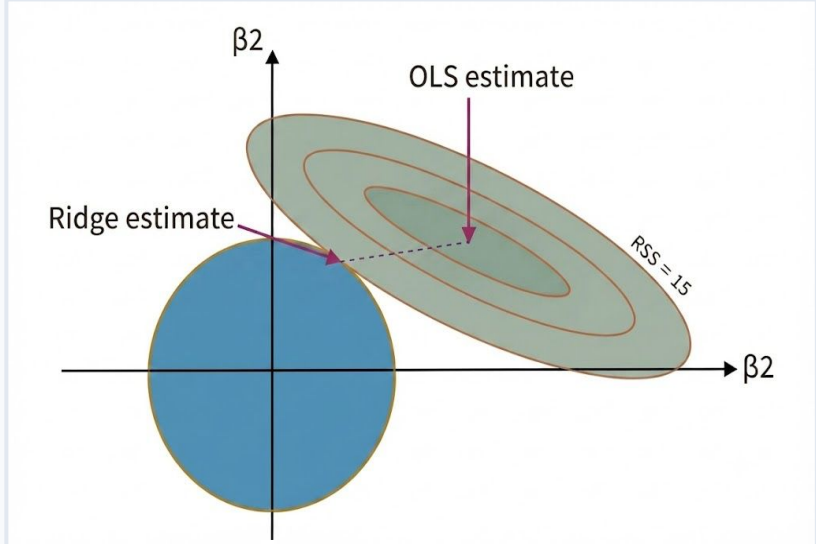
Puts a penalty on big coefficients, so the model favors smaller, more balanced weights

Sharing

Spreads the influence across multiple features instead of one/few features dominate

Lambda

A dial called λ controls the strength, higher for simpler models, lower for simple models.



Ridge Regression

Notebook 03: - Background

Sliding Filters

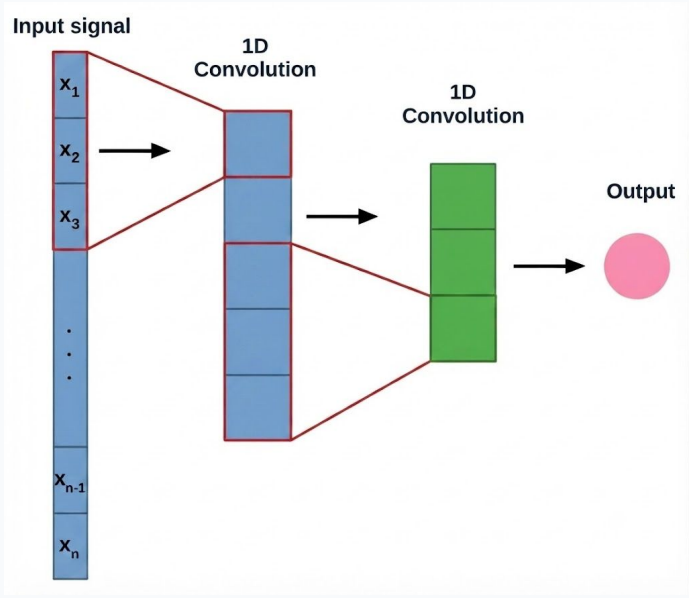
Small filters slide across the sequence and scan local sections.

Batch Normalization

Normalizes layer outputs during training, keeps values stable and learning faster.

Dropout

Randomly turns off neurons during training to prevent overfitting on data.



1D Convolutional Neural Network

Notebook 03: - Background

01. Input (x□)

At each time step, the model reads one input value, x□

02. Update Gate

Decides how much past information to keep versus new information

03. Reset Gate

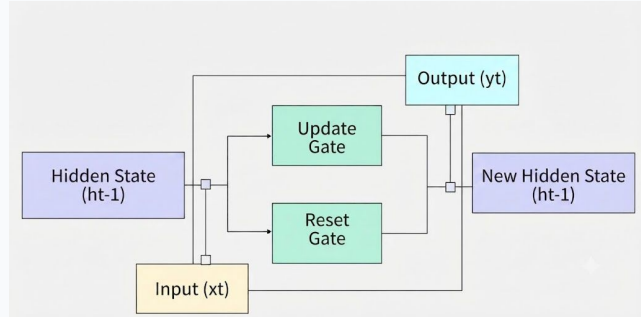
Decides how much past information to forget at each step

04. Hidden State (h□)

Carries memory from past steps and updates at every time step

04. Output (y□)

Produces the output y□ at each time step based on the hidden state h□



Recurrent neural network cell with gating mechanisms

Notebook 03: - Data Preprocessing Strategy

Applied Robust Scaling to reduce impact of spikes and outliers in sensor readings.

Converted tabular telemetry into windowed time-series sequences for temporal learning.

Built a noise-resilient pipeline for stable predictions under volatile appliance usage.

Notebook 03: - Model Architectures Compared

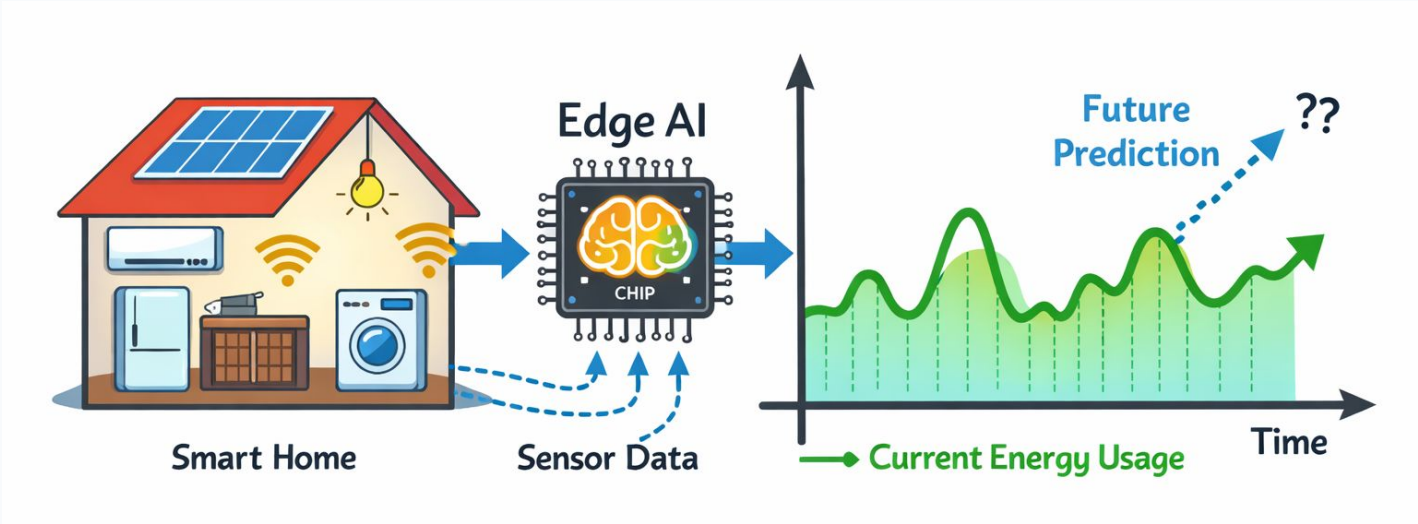
Ridge Regression (Baseline): Linear benchmark on flattened windows for fast, interpretable predictions.

1D-CNN (Edge Model): Learns local temporal patterns in windowed sequences with efficient inference.

GRU (Sequential Model): Captures longer-term temporal dependencies in energy usage.

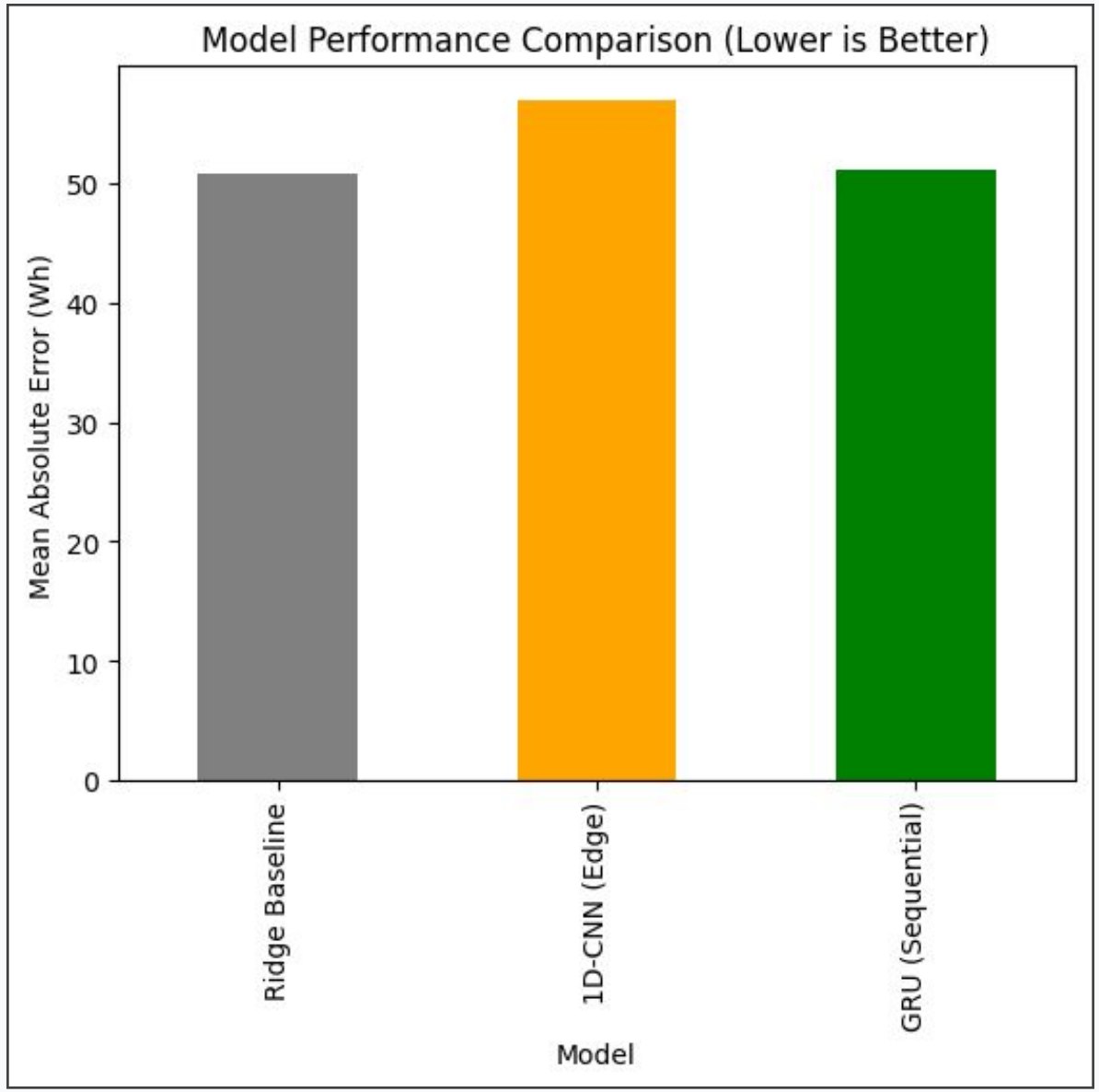
Conclusion:
GRU delivers the most accurate forecasts, outperforming both the linear baseline and CNN on sequential energy patterns.

Schematic Diagram

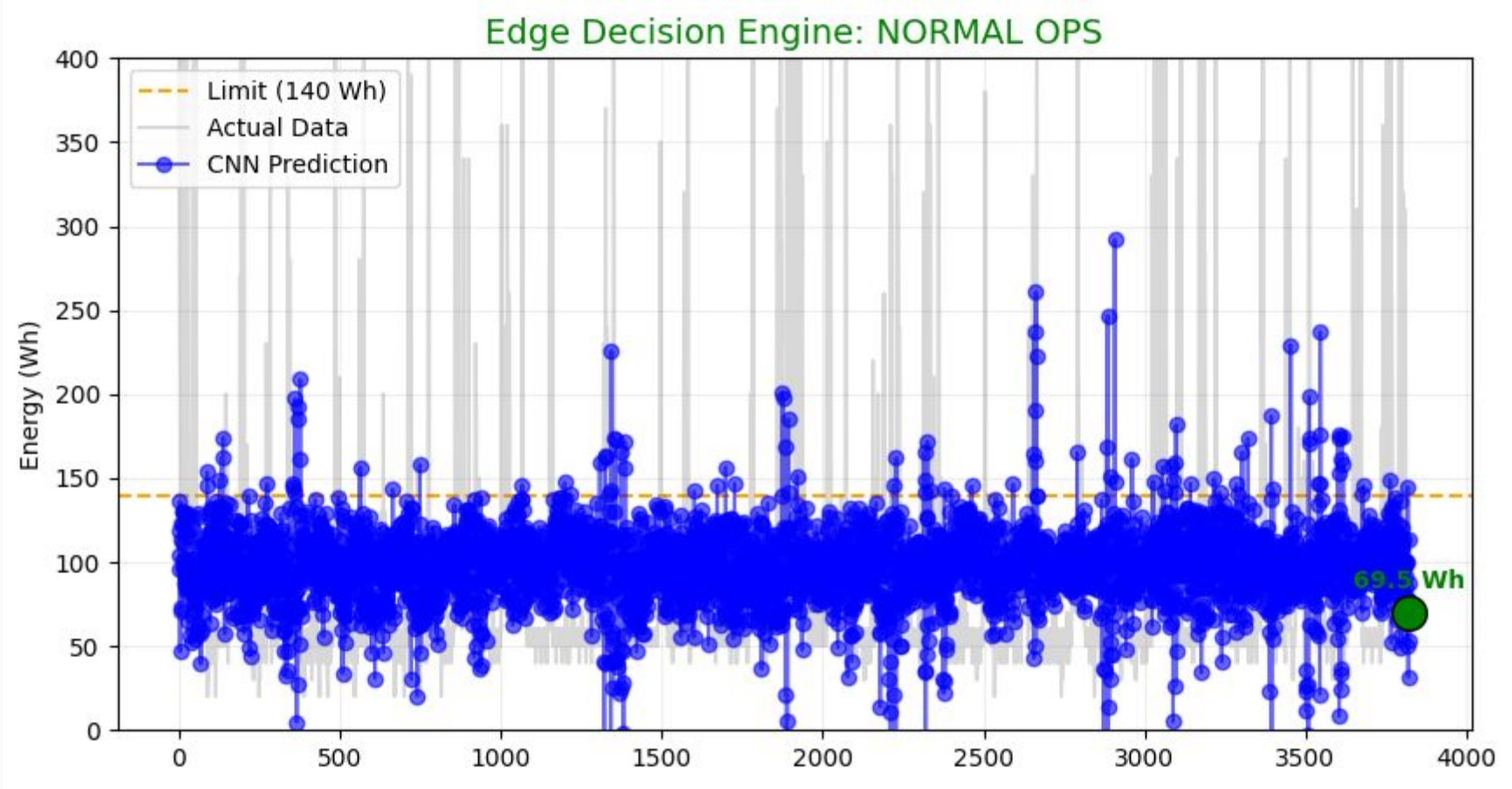


Case Study 03: Schematic Diagram

Notebook 03: - Results and Edge Deployment



Notebook 03: - Results and Edge Deployment



Notebook 03: - Results and Edge Deployment

Model Conversion

Optimizing workflows by converting PyTorch models directly to TorchScript or ONNX formats.

Target Hardware

Tailoring deployment for highly efficient, low-power edge devices like Raspberry Pi.

Future Work

Adding critical temporal features like 'hour-of-day' to capture peak user demand.

Notebook 03: Industrial Use case

Use Case

Microgrid management for large retail centers.

The Problem

Expensive utility penalties during peak energy surges.

Solution

Time-series models predict peaks 15 minutes ahead.

Result

Auto-switches to battery power, saving millions in "peak charges."

https://perspectives.se.com/blog-stream/ai-is-taking-energy-management-to-the-next-frontier

Take It Further: Try This Next

Nixtla: Chronos-2, TimesFM

Foundation models for time series now forecast this case study's appliance-energy problem zero-shot, no custom Ridge, CNN, or GRU training required.

Probabilistic Forecasts

Newer models like Chronos-2 output a full range of likely outcomes, not just one number, exactly what a load-shedding controller needs to weigh its risk.

statsforecast / neuralforecast

Nixtla's open-source libraries wrap dozens of forecasting methods behind one consistent API, so swapping in a new model is a one-line change.

CASE 04

Intelligent Offloading Strategies for Edge AI

Notebook 04: - Objective: Neural Policy Network

Objective & Decision Framework

Primary Goal

Decide if an inference task should be processed Locally or Offloaded.

Decision Factors

Driven by real-time network congestion and local GPU resource availability.

Core Focus

Managing bursty GenAI workloads effectively in hybrid cloud-edge environments.

Notebook 04: - Background

Edge & Cloud Challenges

Resource Constraints

Mobile and edge devices often lack the memory and compute power required to handle large-scale Generative AI workloads locally.

Bursty Workloads

AI inference requests are unpredictable and non-uniform, leading to sudden spikes that can overwhelm standard local hardware resources.

Network Volatility

Relying entirely on cloud infrastructure introduces severe latency risks due to unpredictable real-time network congestion and varying bandwidth.

Notebook 04: - Background

Fit

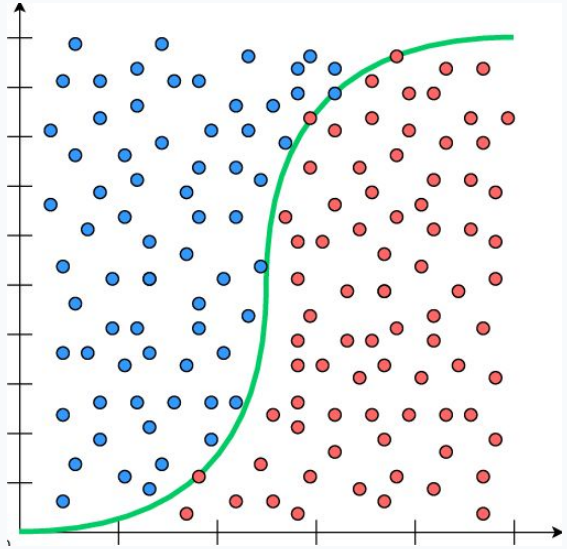
Fits an S-shaped curve that maps inputs to a probability between 0 and 1

Loss

Minimizes cross-entropy (log) loss between predicted probabilities and actual class labels

Coefficients

Weights show how much each feature shifts the log-odds of the positive class



Notebook 04: - Background

Trees

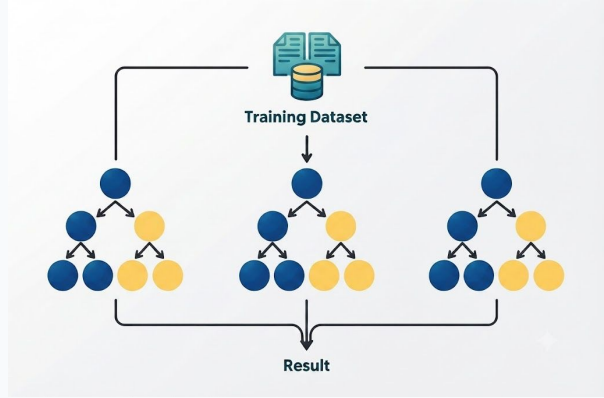
Builds many decision trees instead of just one

Bagging

Each tree trains on a random sample of the data

Voting

Combines all tree predictions for the final answer



Decision Tree

Notebook 04: - Real-World Telemetry

Production Logs

Utilized authentic production logs from the Alibaba Cluster Trace 2026 database.

Key Inputs

Includes critical metrics: Inference Latency, Queue Response Time, and GPU Duty Cycle.

Model Learning

Allowed the neural model to learn adaptively from hardware behavior and network jitter.

Notebook 04: - Solving Class Imbalance

Optimizing Class Representation & Decision Boundaries

The Pitfall

Infrastructure data is often biased toward one decision (e.g., local execution), leading to skewed model training and class imbalance.

The Fix

Implemented Median-Based Cost Scoring to programmatically establish a balanced and fair decision boundary.

The Logic

Label 1 if cost > median (Congested).
Label 0 if cost < median (Efficient) to preserve operational equilibrium.

Notebook 04: - Offloading Strategy Model

Real-Time Neural Network Decisions at the Edge

Architecture

Hybrid PyTorch network utilizing BatchNorm and Dropout for stability.

Goal

Predicts the optimal execution venue (Local vs. Cloud) to minimize total latency.

Adaptability

Engineered for real-time response to dynamic network traffic spikes.

Notebook 04: - Neural vs. Classical Baselines

ML Baselines

Compared against Logistic Regression and Random Forest (100 trees).

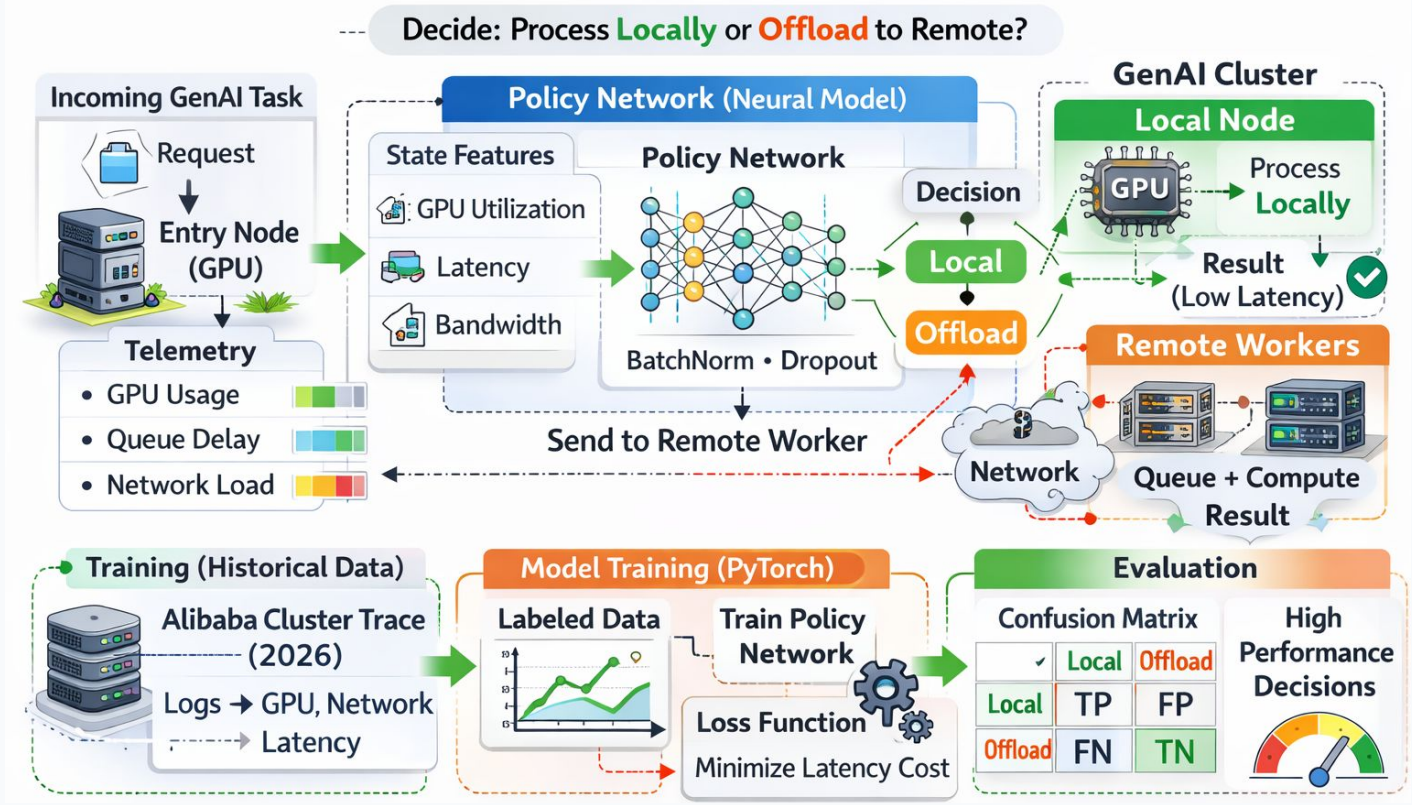
Neural Baseline

Benchmarked with a Simple MLP (16-unit hidden layer) to justify complexity.

Evaluation

Models validated using Cross-Entropy Loss and Confusion Matrices for offloading accuracy.

NB:04 – Schematic Diagram



Case Study 03: Schematic Diagram

Notebook 04: Industrial Use case

Cluster Intelligence (Verizon & AWS)

Use Case

5G-enabled smart traffic monitoring.

The Problem

Cloud latency is too slow for real-time accident alerts.

Solution

Offloads video processing to the nearest 5G cell tower (Edge node).

Result

Latency dropped from 100ms to 10ms, enabling instant emergency response.

Take It Further: Try This Next

vLLM Semantic Router

An open-source router that decides which model and which compute (edge, private, or cloud) should handle each request, the same decision this case study trained a neural policy to make.

Mixture-of-Models

Treating a fleet of specialized models as one system is gaining traction industry-wide, since no single model wins on cost, latency, and quality at once.

Privacy-Aware Routing

Newer research extends offloading decisions to weigh data sensitivity, not just latency and GPU load, deciding what should even be allowed to leave the edge.

Key Performance Lessons

Critical Takeaways for Edge AI and Neural Network Deployment

Edge Abstraction

Feature abstraction (LiDAR binning) is non-negotiable for the edge.

Neural Policies

Data-driven neural policies consistently outperform static schedulers.

Preprocessing

Robust preprocessing is essential when dealing with noisy physical sensors.

Local LLMs

Use a smaller LLM when dealing with local sensitive data.

Technical Comparison of Case Studies

Comparing Architectural Attributes and Constraints Across Core Machine Learning Projects

Case: 01	Case: 02	Case: 03	Case: 04
RAG Chatbot	Imitation Learning	Energy Forecasting	Cluster Offloading
Primary Goal Small Model	**Primary Goal** Sensor-to-Action mapping	**Primary Goal** Time-series prediction	**Primary Goal** Resource Scheduling
Input Data User's document	**Input Data** Binned LiDAR (3 Cones)	**Input Data** Power Telemetry (Amps/Volts)	**Input Data** Cluster Logs (GPU/Network)
Model Type LLM	**Model Type** Lightweight MLP	**Model Type** Sequential (GRU / CNN)	**Model Type** Neural Policy Network
Edge Constraint Limited GPU and Data	**Edge Constraint** Real-time motor latency	**Edge Constraint** Power spikes & Outliers	**Edge Constraint** SLA & Traffic Bursts

Future Directions

Next Steps in Edge Deployment and Feature Optimization

Deployment

Transitioning to edge hardware via TorchScript/ONNX.

Feature Engineering

Enhancing forecasting with temporal metadata.

Sensor Data

Bridging the gap by training on noisy physical sensor data.

Acknowledgements

01

NSF NAIRR Awards

NSF NAIRR-Nat AI Research Resource (NAIRR) Awards OAC-2528533 and OAC-2528534

02

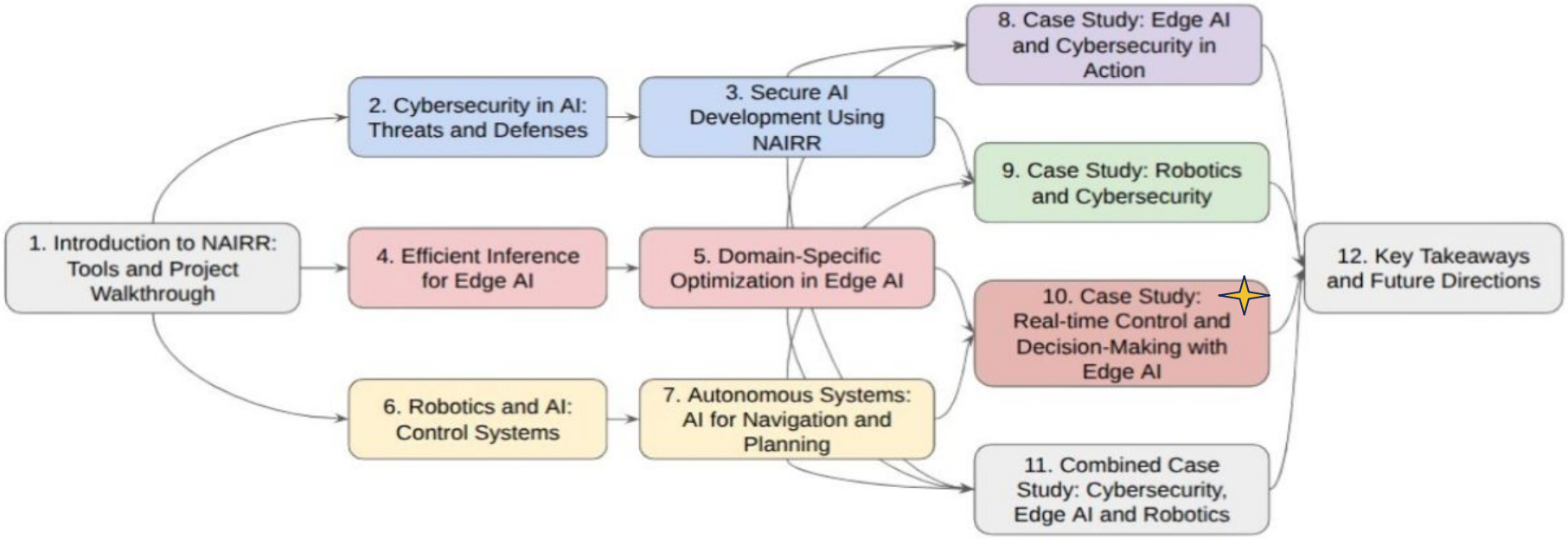
Pilot Allocation

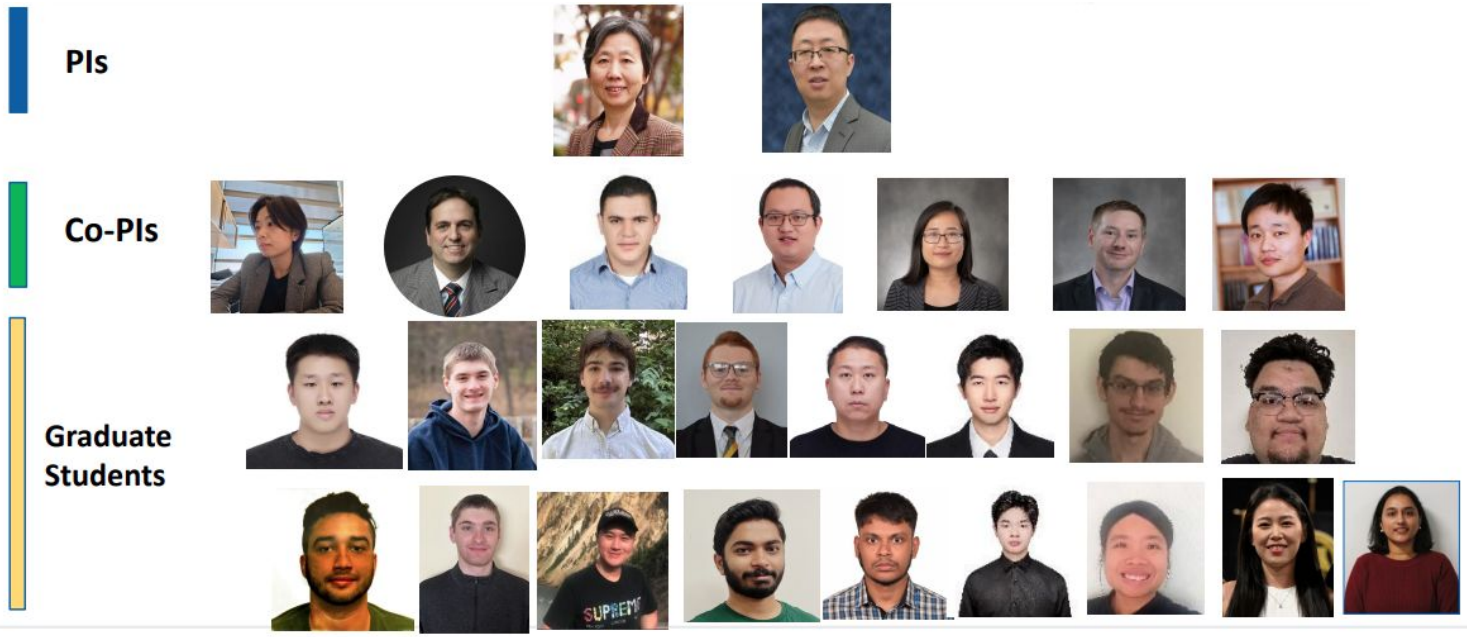
National Artificial Intelligence Research Resource (NAIRR) Pilot allocation NAIRR250048

03

Jetstream2 Cloud

Jetstream2 cloud resource supported by the NSF (award NSF-OAC 2005506) at Indiana University


1. Introduction to NAIRR: Tools and Project Walkthrough
2. Cybersecurity in AI: Threats and Defenses
3. Secure AI Development Using NAIRR
4. Efficient Inference for Edge AI
5. Domain-Specific Optimization in Edge AI
6. Robotics and AI: Control Systems
7. Autonomous Systems: AI for Navigation and Planning
8. Case Study: Edge AI and Cybersecurity in Action
9. Case Study: Robotics and Cybersecurity
10. Case Study: Real-time Control and Decision-Making with Edge AI
11. Combined Case Study: Cybersecurity, Edge AI and Robotics
12. Key Takeaways and Future Directions


PIs
Co-PIs
Graduate Students

Questions?

Thank You.

Any Questions?

Post-workshop Survey

Scan QR code to provide feedback

Agenda

- Motivation: Systems Under Constraints
- The Paradigm Shift to Edge Computing
- Defining Real-Time Control
- The Environmental Constraints
- Intelligent Decision-Making & Offloading
- Transitioning to Case Studies
- Notebook 01: Slides 9-16
- Notebook 02: Slides 17-24
- Notebook 03: Slides 25-32
- Notebook 04:
- Concluding Remarks